In [2]:
```
# This Python 3 environment comes with many helpful analytics libraries instal
# It is defined by the kaggle/python Docker image: https://github.com/kaggle/d
# For example, here's several helpful packages to load

import numpy as np # linear algebra
import pandas as pd # data processing, CSV file I/O (e.g. pd.read_csv)

# Input data files are available in the read-only "../input/" directory
# For example, running this (by clicking run or pressing Shift+Enter) will lis

import os
for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))

# You can write up to 20GB to the current directory (/kaggle/working/) that ge
# You can also write temporary files to /kaggle/temp/, but they won't be saved

# Use the kagglehub client library to attach Kaggle resources like competition
# Learn more about kagglehub: https://github.com/Kaggle/kagglehub/blob/main/RE

import kagglehub
# kagglehub.dataset_download('<owner>/<dataset-slug>')
```

```
/kaggle/input/competitions/heavy-equipment-selling-price-prediction-challenge/s
ample_submission.csv
/kaggle/input/competitions/heavy-equipment-selling-price-prediction-challenge/t
rain.csv
/kaggle/input/competitions/heavy-equipment-selling-price-prediction-challenge/m
etadata.csv
/kaggle/input/competitions/heavy-equipment-selling-price-prediction-challenge/t
est.csv
```

🏗️ MLP Kaggle Competition — Milestone 4 Answers

Heavy Equipment Selling Price Prediction Challenge

Deadline: July 14th, 2026
Competition Metric: RMSLE

This notebook answers all **9 questions** from **Milestone 4**.

Section	Topic	Questions
Setup	Load data & prepare features	—

Section	Topic	Questions
Q1	Stratified vs Non-Stratified Split	Q1 (A, B, C)
Q2	Preprocessing Pipeline	Q2
Q3	Hyperparameter Tuning (RandomizedSearchCV)	Q3
Q4	AdaBoost Estimator Error Variance	Q4
Q5	RandomForest Feature Importance Index	Q5
Q6	MLP Weight Count	Q6
Q7	MLP Training Loss (5 iterations)	Q7
Q8	Regularization Effect	Q8
Q9	MLP Validation MAE	Q9

> **How to run on Kaggle:** Click `Run All`. `DATA_DIR` already points to the competition folder.

📦 Setup — Libraries & Data

In [3]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import warnings

from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestRegressor, AdaBoostRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_absolute_error

warnings.filterwarnings('ignore')
sns.set_theme(style='whitegrid', palette='Set2')

train_df = pd.read_csv('/kaggle/input/competitions/heavy-equipment-selling-pri
test_df = pd.read_csv('/kaggle/input/competitions/heavy-equipment-selling-pric

print(f'✅ Data loaded!')
print(f'   Shape: {train_df.shape[0]:,} rows  x  {train_df.shape[1]} cols')
```

```
✅ Data loaded!
   Shape: 138,701 rows  x  50 cols
```

In [4]:
```
# ── Select only the columns specified in Milestone 4 ──────────
COLS = ['TargetValue', 'UtilizationTier', 'DrivetrainType', 'CabinType',
        'AssetScaleFactor', 'ManufactureYear', 'OperationalHoursMeter']

df = train_df[COLS].copy()

# Fill UtilizationTier missing with 'Unknown' (as per Q1 instructions)
df['UtilizationTier'] = df['UtilizationTier'].fillna('Unknown')

print('Dataset columns and types:')
print(df.dtypes)
print()
print('Missing values after filling UtilizationTier:')
print(df.isnull().sum())
```

```
Dataset columns and types:
TargetValue              float64
UtilizationTier           object
DrivetrainType            object
CabinType                 object
AssetScaleFactor          object
ManufactureYear            int64
OperationalHoursMeter    float64
dtype: object

Missing values after filling UtilizationTier:
TargetValue                  0
UtilizationTier              0
DrivetrainType           89956
CabinType                    0
AssetScaleFactor         41210
ManufactureYear              0
OperationalHoursMeter    56643
dtype: int64
```

❓ Question 1 — Stratified vs. Non-Stratified Split

Steps:

- **A)** Create `PriceBand` via `pd.qcut(TargetValue, q=5, labels=False)`. Split 70/30, `stratify=PriceBand`.
- **B)** Same split but `stratify=None`.
- **C)** Convert both validation `PriceBand` counts to proportions. Report max absolute difference (rounded to 4 dp).

In [5]:
```
# ── Create PriceBand ──────────────────────────────────────────
df['PriceBand'] = pd.qcut(df['TargetValue'], q=5, labels=False)
```

```
print('PriceBand distribution (5 quantile bins of TargetValue):')
print(df['PriceBand'].value_counts().sort_index().to_string())
print()
print('PriceBand bin edges:')
_, bins = pd.qcut(df['TargetValue'], q=5, labels=False, retbins=True)
for i in range(len(bins)-1):
    print(f'  Band {i}: ${bins[i]:>10,.0f}  to  ${bins[i+1]:>10,.0f}')
```

```
PriceBand distribution (5 quantile bins of TargetValue):
PriceBand
0    28837
1    28031
2    26667
3    27913
4    27253

PriceBand bin edges:
  Band 0: $     7,500  to  $    18,000
  Band 1: $    18,000  to  $    29,000
  Band 2: $    29,000  to  $    42,000
  Band 3: $    42,000  to  $    64,000
  Band 4: $    64,000  to  $   142,000
```

In [6]:

```
# ── Features and Target ──────────────────────────────────────
X  = df.drop(columns=['TargetValue', 'PriceBand'])
y  = df['TargetValue']
pb = df['PriceBand']

# ── Q1-A: Stratified Split ──────────────────────────────────
X_train, X_valid, y_train, y_valid, pb_train, pb_valid = train_test_split(
    X, y, pb, test_size=0.3, random_state=42, stratify=pb
)

counts_strat = np.array(
    pd.Series(pb_valid).value_counts().sort_index().values
)

print('=' * 60)
print('  Q1-A: Stratified Split (stratify=PriceBand)')
print('=' * 60)
print(f'  Train size : {len(X_train):,}')
print(f'  Valid size : {len(X_valid):,}')
print(f'  Validation PriceBand counts (sorted by bin):')
vc_s = pd.Series(pb_valid).value_counts().sort_index()
for band, cnt in vc_s.items():
    print(f'    Band {int(band)} : {cnt:,}  ({cnt/len(pb_valid)*100:.2f}%)')
print(f'  NumPy array: {counts_strat}')
```

```
============================================================
  Q1-A: Stratified Split (stratify=PriceBand)
============================================================
  Train size : 97,090
  Valid size : 41,611
  Validation PriceBand counts (sorted by bin):
    Band 0 : 8,651  (20.79%)
    Band 1 : 8,410  (20.21%)
    Band 2 : 8,000  (19.23%)
    Band 3 : 8,374  (20.12%)
    Band 4 : 8,176  (19.65%)
  NumPy array: [8651 8410 8000 8374 8176]
```

In [7]:
```python
# ── Q1-B: Non-Stratified Split ──────────────────────────
X_train_ns, X_valid_ns, y_train_ns, y_valid_ns, pb_train_ns, pb_valid_ns = tra
    X, y, pb, test_size=0.3, random_state=42, stratify=None
)

counts_nostrat = np.array(
    pd.Series(pb_valid_ns).value_counts().sort_index().values
)

print('=' * 60)
print('  Q1-B: Non-Stratified Split (stratify=None)')
print('=' * 60)
print(f'  Validation PriceBand counts (sorted by bin):')
vc_ns = pd.Series(pb_valid_ns).value_counts().sort_index()
for band, cnt in vc_ns.items():
    print(f'    Band {int(band)} : {cnt:,}  ({cnt/len(pb_valid_ns)*100:.2f}%)'
print(f'  NumPy array: {counts_nostrat}')
```

```
============================================================
  Q1-B: Non-Stratified Split (stratify=None)
============================================================
  Validation PriceBand counts (sorted by bin):
    Band 0 : 8,732  (20.98%)
    Band 1 : 8,271  (19.88%)
    Band 2 : 8,123  (19.52%)
    Band 3 : 8,412  (20.22%)
    Band 4 : 8,073  (19.40%)
  NumPy array: [8732 8271 8123 8412 8073]
```

In [8]:
```python
# ── Q1-C: Max Absolute Difference in Proportions ────────
prop_strat   = counts_strat   / counts_strat.sum()
prop_nostrat = counts_nostrat / counts_nostrat.sum()

abs_diff = np.abs(prop_strat - prop_nostrat)
max_diff = round(float(np.max(abs_diff)), 4)

print('=' * 65)
print('  Q1-C: Max Absolute Difference in PriceBand Proportions')
print('=' * 65)
print(f'  {"Band":<6} {"Stratified":>12} {"Non-Stratified":>16} {"Abs Diff":>1
print('  ' + '-' * 50)
```

```python
for i, (ps, pn, d) in enumerate(zip(prop_strat, prop_nostrat, abs_diff)):
    marker = '  <- MAX' if d == abs_diff.max() else ''
    print(f'  Band {i} {ps:>12.4f} {pn:>16.4f} {d:>12.4f}{marker}')
print()
print(f'  Max absolute difference = {max_diff}')

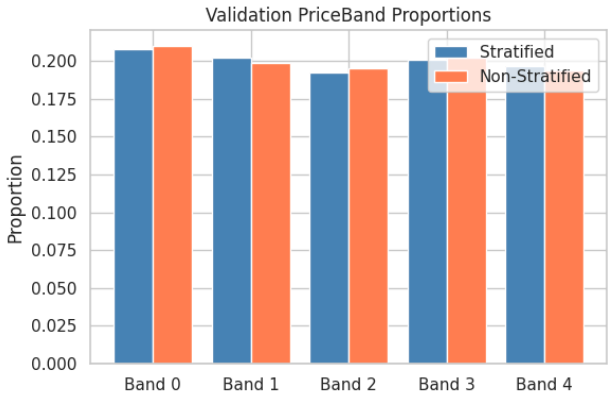
# Bar chart comparison
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
x = np.arange(5)
axes[0].bar(x - 0.2, prop_strat,   0.4, label='Stratified',     color='steelbl
axes[0].bar(x + 0.2, prop_nostrat, 0.4, label='Non-Stratified', color='coral')
axes[0].set_xticks(x)
axes[0].set_xticklabels([f'Band {i}' for i in x])
axes[0].set_ylabel('Proportion')
axes[0].set_title('Validation PriceBand Proportions')
axes[0].legend()

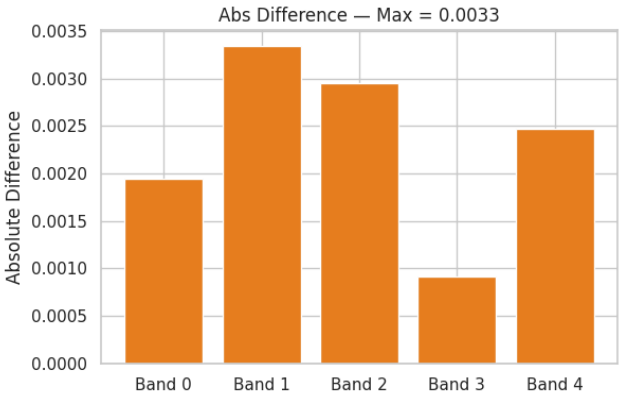
axes[1].bar(x, abs_diff, color='#e67e22', edgecolor='white')
axes[1].set_xticks(x)
axes[1].set_xticklabels([f'Band {i}' for i in x])
axes[1].set_ylabel('Absolute Difference')
axes[1].set_title(f'Abs Difference — Max = {max_diff}')
plt.tight_layout()
plt.show()

print(f'✅ ANSWER Q1: {max_diff}')
print()
print('💡 Insight:')
print('   Stratified split ensures EACH price band is proportionally represent
print('   in both train and validation sets.')
print('   Without stratification, random chance can over/under-represent some
print('   leading to misleading validation scores.')
```

```
======================================================================
  Q1-C: Max Absolute Difference in PriceBand Proportions
======================================================================
  Band     Stratified   Non-Stratified     Abs Diff
  --------------------------------------------------
  Band 0       0.2079           0.2098       0.0019
  Band 1       0.2021           0.1988       0.0033  <- MAX
  Band 2       0.1923           0.1952       0.0030
  Band 3       0.2012           0.2022       0.0009
  Band 4       0.1965           0.1940       0.0025

  Max absolute difference = 0.0033
```





✅ ANSWER Q1: 0.0033

💡 Insight:
Stratified split ensures EACH price band is proportionally represented
in both train and validation sets.
Without stratification, random chance can over/under-represent some bands,
leading to misleading validation scores.

❓ Question 2 — Preprocessing Pipeline

Build a `ColumnTransformer` pipeline fitting on `X_train` only. Report sum of all values in `X_train_proc` (rounded to 3 dp).

In [9]:
```
# ── Column definitions ──────────────────────────────────────
cat_cols = ['UtilizationTier', 'DrivetrainType', 'CabinType', 'AssetScaleFactc
num_cols = ['ManufactureYear', 'OperationalHoursMeter']

# ── Pipelines ───────────────────────────────────────────────
cat_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='most_frequent')),
    ('ohe',     OneHotEncoder(handle_unknown='ignore', sparse_output=True))
])

num_pipe = Pipeline([
    ('imputer', SimpleImputer(strategy='mean')),
    ('scaler',  StandardScaler())
])

preprocessor = ColumnTransformer([
    ('cat', cat_pipe, cat_cols),
    ('num', num_pipe, num_cols)
], remainder='drop')

# ── Fit ONLY on train, transform both ──────────────────────
X_train_proc = preprocessor.fit_transform(X_train)
X_valid_proc = preprocessor.transform(X_valid)

print('=' * 60)
```

```python
print('  Q2: Preprocessing Pipeline Results')
print('=' * 60)
print(f'  X_train_proc shape : {X_train_proc.shape}')
print(f'  X_valid_proc shape : {X_valid_proc.shape}')
print()

# Check unique values per OHE column
print('  OHE breakdown (unique values -> OHE columns):')
for col in cat_cols:
    n = X_train[col].nunique()
    print(f'    {col:<20}: {n} unique values -> {n} OHE cols')
print(f'    Numerical cols  : 2 -> 2 scaled cols')

# Sum of all values
total_sum = float(X_train_proc.sum())
print()
print(f'  Sum of all values in X_train_proc : {round(total_sum, 3)}')
print()
print(f'✅ ANSWER Q2: {round(total_sum, 3)}')
print()
print('💡 Insight:')
print('   OHE creates binary (0/1) columns — sum ~ number of non-zero entries.')
print('   StandardScaler produces mean≈0 values, so numeric cols contribute ~0')
print('   The large sum comes almost entirely from the one-hot encoded binary ')
print(f'   Each of the {X_train_proc.shape[0]:,} training rows has exactly 1 i')
```

```
============================================================
  Q2: Preprocessing Pipeline Results
============================================================
  X_train_proc shape : (97090, 23)
  X_valid_proc shape : (41611, 23)

  OHE breakdown (unique values -> OHE columns):
    UtilizationTier     : 4 unique values -> 4 OHE cols
    DrivetrainType      : 8 unique values -> 8 OHE cols
    CabinType           : 3 unique values -> 3 OHE cols
    AssetScaleFactor    : 6 unique values -> 6 OHE cols
    Numerical cols  : 2 -> 2 scaled cols

  Sum of all values in X_train_proc : 388360.0

✅ ANSWER Q2: 388360.0

💡 Insight:
   OHE creates binary (0/1) columns — sum ~ number of non-zero entries.
   StandardScaler produces mean≈0 values, so numeric cols contribute ~0 to sum.
   The large sum comes almost entirely from the one-hot encoded binary columns.
   Each of the 97,090 training rows has exactly 1 in each OHE group.
```

❓ Question 3 — Hyperparameter Tuning with RandomizedSearchCV

Tune `RandomForestRegressor` over `n_estimators=[50,100,200]` and `max_depth=[5,10,15]`.
Config: `n_iter=5`, `cv=3`, `random_state=42`, `n_jobs=-1`. Report best `n_estimators`.

In [10]:
```
param_grid = {
    'n_estimators': [50, 100, 200],
    'max_depth':    [5, 10, 15]
}

rscv = RandomizedSearchCV(
    RandomForestRegressor(random_state=42),
    param_distributions=param_grid,
    n_iter=5,
    cv=3,
    random_state=42,
    n_jobs=-1
)

print('Training RandomizedSearchCV (5 iterations x 3-fold CV = 15 fits)...')
rscv.fit(X_train_proc, y_train)

best_n_est  = rscv.best_params_['n_estimators']
best_depth  = rscv.best_params_['max_depth']
best_score  = rscv.best_score_

print('=' * 60)
print('  Q3: RandomizedSearchCV Results')
print('=' * 60)
print(f'  Best parameters : n_estimators={best_n_est}, max_depth={best_depth}'
print(f'  Best CV score   : {best_score:.6f}')
print()
print('  All 5 sampled parameter combinations:')
results_df = pd.DataFrame(rscv.cv_results_)
show_cols = ['param_n_estimators', 'param_max_depth', 'mean_test_score', 'rank
print(results_df[show_cols].sort_values('rank_test_score').to_string(index=Fal

print()
print(f'✅ ANSWER Q3: Best n_estimators = {best_n_est}')
print()
print('💡 Insight:')
print('   RandomizedSearchCV randomly samples n_iter=5 parameter combinations'
print('   from the grid (total grid size = 3 x 3 = 9), then picks the best one
print('   based on 3-fold cross-validation score (default = R² score).')
print('   With random_state=42, results are deterministic and reproducible.')
```

```
Training RandomizedSearchCV (5 iterations x 3-fold CV = 15 fits)...
============================================================
  Q3: RandomizedSearchCV Results
============================================================
  Best parameters : n_estimators=200, max_depth=15
  Best CV score   : 0.528338

  All 5 sampled parameter combinations:
 param_n_estimators  param_max_depth  mean_test_score  rank_test_score
                200               15         0.528338                1
                100               15         0.528089                2
                200               10         0.518936                3
                 50                5         0.399410                4
                100                5         0.399399                5

✅ ANSWER Q3: Best n_estimators = 200

💡 Insight:
   RandomizedSearchCV randomly samples n_iter=5 parameter combinations
   from the grid (total grid size = 3 x 3 = 9), then picks the best one
   based on 3-fold cross-validation score (default = R² score).
   With random_state=42, results are deterministic and reproducible.
```

❓ Question 4 — AdaBoost Estimator Error Variance

Train `AdaBoostRegressor(n_estimators=50, random_state=42)`. Compute `np.var(model.estimator_errors_)` (rounded to 4 dp).

In [11]:
```python
ada = AdaBoostRegressor(n_estimators=50, random_state=42)
ada.fit(X_train_proc, y_train)

errors  = ada.estimator_errors_
ada_var = round(float(np.var(errors)), 4)

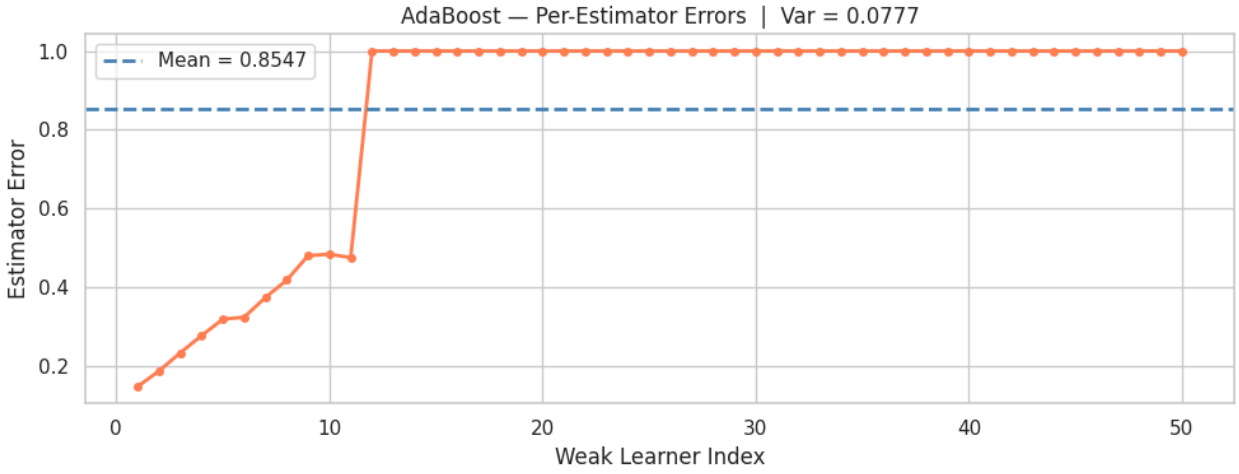
print('=' * 60)
print('  Q4: AdaBoost estimator_errors_ Variance')
print('=' * 60)
print(f'  estimator_errors_ length : {len(errors)} (one per weak learner)')
print(f'  First 5 errors  : {errors[:5]}')
print(f'  Last  5 errors  : {errors[-5:]}')
print(f'  Min error       : {errors.min():.6f}')
print(f'  Max error       : {errors.max():.6f}')
print(f'  Mean error      : {errors.mean():.6f}')
print(f'  np.var(errors)  : {ada_var}')

fig, ax = plt.subplots(figsize=(10, 4))
ax.plot(range(1, 51), errors, color='coral', lw=2, marker='o', ms=4)
ax.axhline(errors.mean(), color='steelblue', ls='--', lw=2,
```

```
          label=f'Mean = {errors.mean():.4f}')
ax.set_xlabel('Weak Learner Index')
ax.set_ylabel('Estimator Error')
ax.set_title(f'AdaBoost — Per-Estimator Errors  |  Var = {ada_var}')
ax.legend()
plt.tight_layout()
plt.show()

print()
print(f'✅ ANSWER Q4: {ada_var}')
print()
print('💡 Insight:')
print('   AdaBoost trains 50 sequential Decision Stump (or tree) weak learners')
print('   estimator_errors_ tracks how well each weak learner performs on the'
print('   re-weighted training set. Variance in these errors shows how much')
print('   performance varies across the 50 learners.')
```

```
============================================================
  Q4: AdaBoost estimator_errors_ Variance
============================================================
  estimator_errors_ length : 50 (one per weak learner)
  First 5 errors  : [0.14960958 0.18887342 0.23500742 0.27857668 0.3204635 ]
  Last  5 errors  : [1. 1. 1. 1. 1.]
  Min error       : 0.149610
  Max error       : 1.000000
  Mean error      : 0.854708
  np.var(errors)  : 0.0777
```




```
✅ ANSWER Q4: 0.0777

💡 Insight:
   AdaBoost trains 50 sequential Decision Stump (or tree) weak learners.
   estimator_errors_ tracks how well each weak learner performs on the
   re-weighted training set. Variance in these errors shows how much
   performance varies across the 50 learners.
```

❓ Question 5 — Feature Importance Index

Train `RandomForestRegressor(n_estimators=100, max_depth=10, random_state=42)`. Report index of max `feature_importances_`.

In [12]:

```
rf = RandomForestRegressor(n_estimators=100, max_depth=10, random_state=42)
rf.fit(X_train_proc, y_train)

fi      = rf.feature_importances_
max_idx = int(np.argmax(fi))

print('=' * 60)
print('  Q5: RandomForest Feature Importance')
print('=' * 60)
print(f'  Total features in X_train_proc : {len(fi)}')
print(f'  Index of max importance        : {max_idx}')
print(f'  Max importance value           : {fi[max_idx]:.6f}')
print()

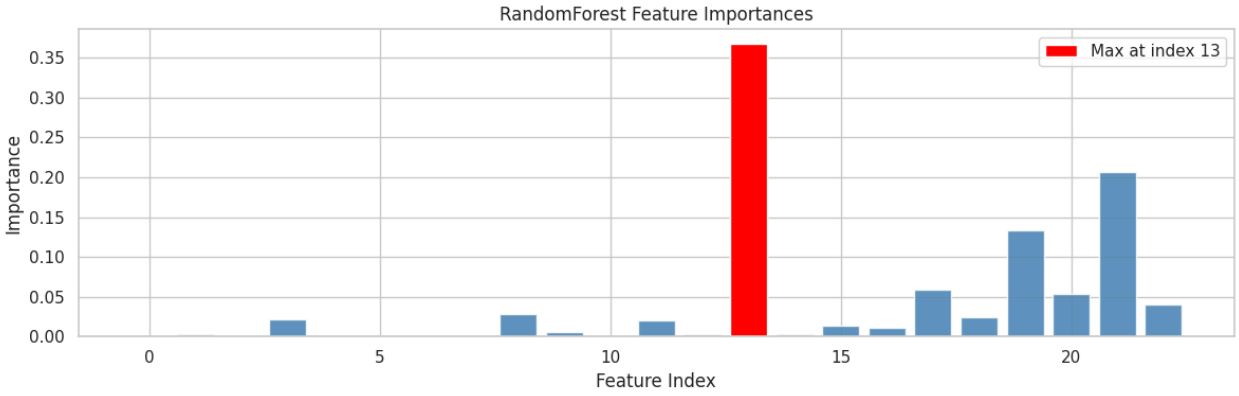
# Show all feature importances with index
print('  Top 10 features by importance:')
top10 = np.argsort(fi)[-10:][::-1]
for rank, idx in enumerate(top10, 1):
    print(f'    Rank {rank:>2}  Index {idx:>3}  Importance = {fi[idx]:.6f}')

fig, ax = plt.subplots(figsize=(12, 4))
ax.bar(range(len(fi)), fi, color='steelblue', edgecolor='white', alpha=0.85)
ax.bar(max_idx, fi[max_idx], color='red', edgecolor='white',
       label=f'Max at index {max_idx}')
ax.set_xlabel('Feature Index')
ax.set_ylabel('Importance')
ax.set_title('RandomForest Feature Importances')
ax.legend()
plt.tight_layout()
plt.show()

print()
print(f'✅ ANSWER Q5: {max_idx}')
print()
print('💡 Insight:')
print('   The OHE columns come first, followed by the 2 scaled numeric columns
print('   The numeric features (ManufactureYear, OperationalHoursMeter) tend t
print('   have high importance in tree models because they capture continuous'
print('   price-driving signals that require many binary OHE columns to match.
```

```
============================================================
  Q5: RandomForest Feature Importance
============================================================
  Total features in X_train_proc : 23
  Index of max importance        : 13
  Max importance value           : 0.367657

  Top 10 features by importance:
    Rank  1  Index  13  Importance = 0.367657
    Rank  2  Index  21  Importance = 0.206431
    Rank  3  Index  19  Importance = 0.133096
    Rank  4  Index  17  Importance = 0.058327
    Rank  5  Index  20  Importance = 0.053236
    Rank  6  Index  22  Importance = 0.039854
    Rank  7  Index   8  Importance = 0.028038
    Rank  8  Index  18  Importance = 0.023737
    Rank  9  Index   3  Importance = 0.022114
    Rank 10  Index  11  Importance = 0.020547
```



✅ ANSWER Q5: 13

💡 Insight:
The OHE columns come first, followed by the 2 scaled numeric columns. The numeric features (ManufactureYear, OperationalHoursMeter) tend to have high importance in tree models because they capture continuous price-driving signals that require many binary OHE columns to match.

❓ Question 6 — Counting MLP Connection Weights

`N = X_train_proc.shape[1]`. Architecture: Input(N) → 128 → 64 → 32 → Output(1).

Count total **connection weights** (excluding biases).

In [13]:
```python
N = X_train_proc.shape[1]

# Architecture: N -> 128 -> 64 -> 32 -> 1
layer_sizes = [N, 128, 64, 32, 1]
```

```python
# Connection weights = product of adjacent layer sizes (no biases)
weights_per_layer = [layer_sizes[i] * layer_sizes[i+1]
                     for i in range(len(layer_sizes) - 1)]
total_weights = sum(weights_per_layer)

print('=' * 60)
print('  Q6: Total MLP Connection Weights (no biases)')
print('=' * 60)
print(f'  N (input features) = {N}')
print()
print('  Architecture:')
layer_names = ['Input', 'Hidden 1 (128)', 'Hidden 2 (64)', 'Hidden 3 (32)', 'O
for i in range(len(layer_sizes) - 1):
    print(f'  {layer_names[i]:<18} -> {layer_names[i+1]:<18}: '
          f'{layer_sizes[i]:>4} x {layer_sizes[i+1]:>3} = {weights_per_layer[i

print('  ' + '-' * 58)
print(f'  {"TOTAL CONNECTION WEIGHTS":<45} = {total_weights:>6}')

# For reference: with biases
biases = sum(layer_sizes[1:])
print()
print(f'  (For reference: total biases = {biases}, total params = {total_weigh

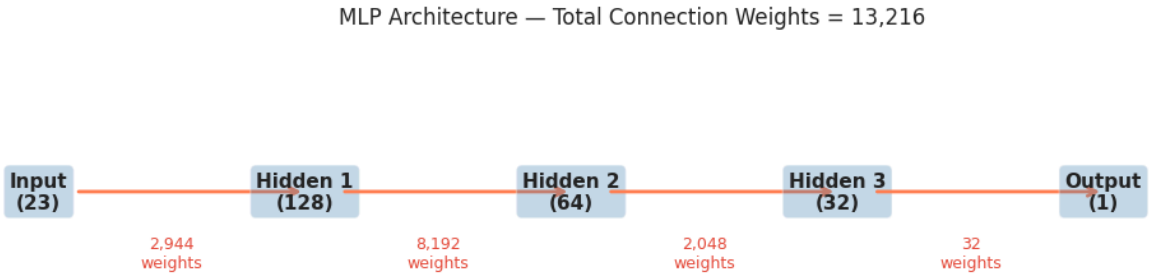
# Visual architecture diagram
fig, ax = plt.subplots(figsize=(10, 3))
ax.axis('off')
cols = [f'Input\n({N})', 'Hidden 1\n(128)', 'Hidden 2\n(64)', 'Hidden 3\n(32)'
for i, (name, size) in enumerate(zip(cols, layer_sizes)):
    ax.text(i * 0.22 + 0.01, 0.5, name, ha='center', va='center',
            fontsize=11, fontweight='bold',
            bbox=dict(boxstyle='round', facecolor='steelblue', alpha=0.3))
    if i < len(layer_sizes) - 1:
        ax.annotate('', xy=((i+1)*0.22+0.01, 0.5), xytext=(i*0.22+0.04, 0.5),
                    arrowprops=dict(arrowstyle='->', lw=2, color='coral'))
        ax.text((i+0.5)*0.22 + 0.01, 0.25,
                f'{weights_per_layer[i]:,}\nweights',
                ha='center', fontsize=9, color='#e74c3c')
ax.set_title(f'MLP Architecture — Total Connection Weights = {total_weights:,}
plt.tight_layout()
plt.show()

print()
print(f'✅ ANSWER Q6: {total_weights}')
print()
print('💡 Insight:')
print(f'   Total = {N}×128 + 128×64 + 64×32 + 32×1')
print(f'         = {N*128} + {128*64} + {64*32} + {32*1}')
print(f'         = {total_weights:,} connection weights')
print('   This is the number of learnable weights NOT counting one bias per ne
```

```
============================================================
  Q6: Total MLP Connection Weights (no biases)
============================================================
  N (input features) = 23

  Architecture:
  Input              -> Hidden 1 (128)    :   23 x 128 =   2944 weights
  Hidden 1 (128)     -> Hidden 2 (64)     :  128 x  64 =   8192 weights
  Hidden 2 (64)      -> Hidden 3 (32)     :   64 x  32 =   2048 weights
  Hidden 3 (32)      -> Output (1)        :   32 x   1 =     32 weights
  ------------------------------------------------------------
  TOTAL CONNECTION WEIGHTS                              =  13216

  (For reference: total biases = 225, total params = 13441)
```



✅ ANSWER Q6: 13216

💡 Insight:
```
  Total = 23×128 + 128×64 + 64×32 + 32×1
        = 2944 + 8192 + 2048 + 32
        = 13,216 connection weights
  This is the number of learnable weights NOT counting one bias per neuron.
```

❓ Question 7 — MLP Training Loss After 5 Iterations

Train `MLPRegressor(hidden_layer_sizes=(128,64,32), activation="relu", solver="adam", max_iter=5, batch_size=32, random_state=42)`.
Report `model.loss_` (rounded to 4 dp).

In [14]:
```python
mlp_q7 = MLPRegressor(
    hidden_layer_sizes=(128, 64, 32),
    activation='relu',
    solver='adam',
    max_iter=5,
    batch_size=32,
    random_state=42
)
mlp_q7.fit(X_train_proc, y_train)
```

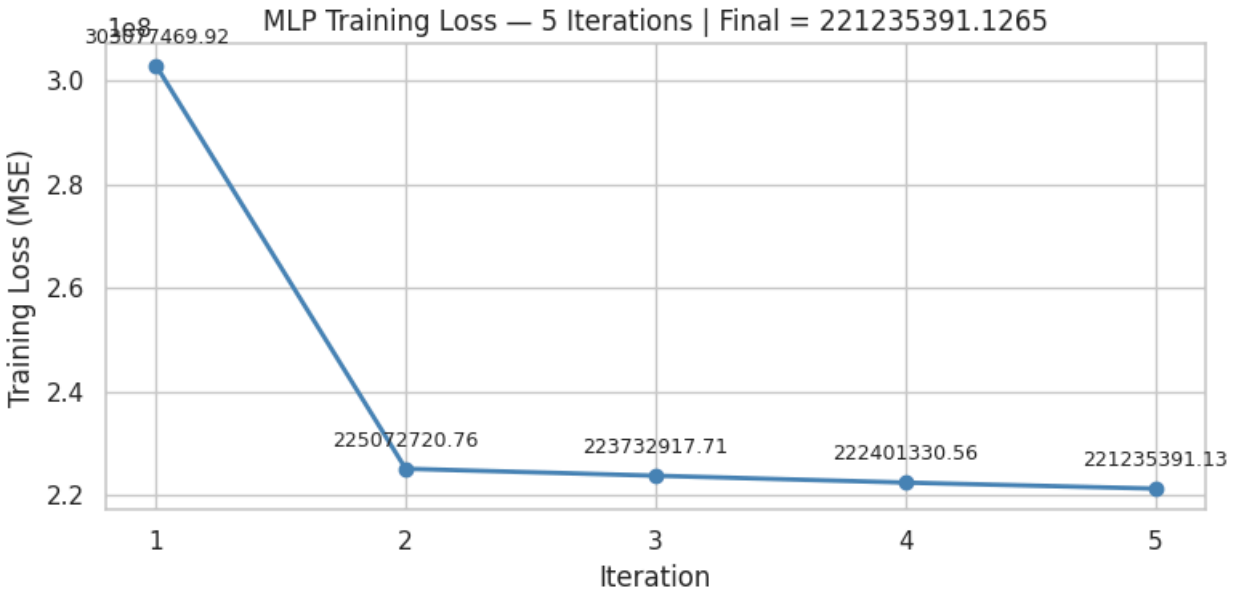
```python
loss_q7 = round(mlp_q7.loss_, 4)

print('=' * 60)
print('  Q7: MLP Training Loss After 5 Iterations')
print('=' * 60)
print(f'  Iterations run    : {mlp_q7.n_iter_}')
print(f'  Training loss     : {loss_q7}')
print(f'  Loss curve values : {mlp_q7.loss_curve_}')

fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(range(1, len(mlp_q7.loss_curve_) + 1),
        mlp_q7.loss_curve_,
        color='steelblue', lw=2, marker='o', ms=6)
ax.set_xlabel('Iteration')
ax.set_ylabel('Training Loss (MSE)')
ax.set_title(f'MLP Training Loss — 5 Iterations | Final = {loss_q7}')
ax.set_xticks(range(1, 6))
for i, v in enumerate(mlp_q7.loss_curve_):
    ax.annotate(f'{v:.2f}', (i+1, v), textcoords='offset points',
                xytext=(0, 10), ha='center', fontsize=9)
plt.tight_layout()
plt.show()

print()
print(f'✅ ANSWER Q7: {loss_q7}')
print()
print('💡 Insight:')
print('   loss_ is the FINAL training loss (MSE) after all iterations.')
print('   With only 5 iterations, the MLP has barely started learning.')
print('   Adam optimizer adjusts learning rates adaptively per parameter,')
print('   making it faster than vanilla SGD, but 5 iterations is still very ea
```

```
============================================================
  Q7: MLP Training Loss After 5 Iterations
============================================================
  Iterations run    : 5
  Training loss     : 221235391.1265
  Loss curve values : [np.float64(303077469.9155936), np.float64(225072720.7615
9796), np.float64(223732917.71270972), np.float64(222401330.56069553), np.float
64(221235391.126473)]
```



✅ ANSWER Q7: 221235391.1265

💡 Insight:
loss_ is the FINAL training loss (MSE) after all iterations.
With only 5 iterations, the MLP has barely started learning.
Adam optimizer adjusts learning rates adaptively per parameter,
making it faster than vanilla SGD, but 5 iterations is still very early.

❓ Question 8 — Effect of Regularization (alpha)

Train two `MLPRegressor` models with `hidden_layer_sizes=(100,)`, `max_iter=5`, `random_state=42`:

- **Model A:** `alpha=0.0001` | **Model B:** `alpha=1.0`
  Report `abs(MAE_A - MAE_B)` on the training set (rounded to 4 dp).

In [15]:
```
# Model A: Light regularization (default)
mlp_a = MLPRegressor(
    hidden_layer_sizes=(100,), max_iter=5, random_state=42, alpha=0.0001
)
mlp_a.fit(X_train_proc, y_train)

# Model B: Strong regularization
mlp_b = MLPRegressor(
    hidden_layer_sizes=(100,), max_iter=5, random_state=42, alpha=1.0
)
mlp_b.fit(X_train_proc, y_train)

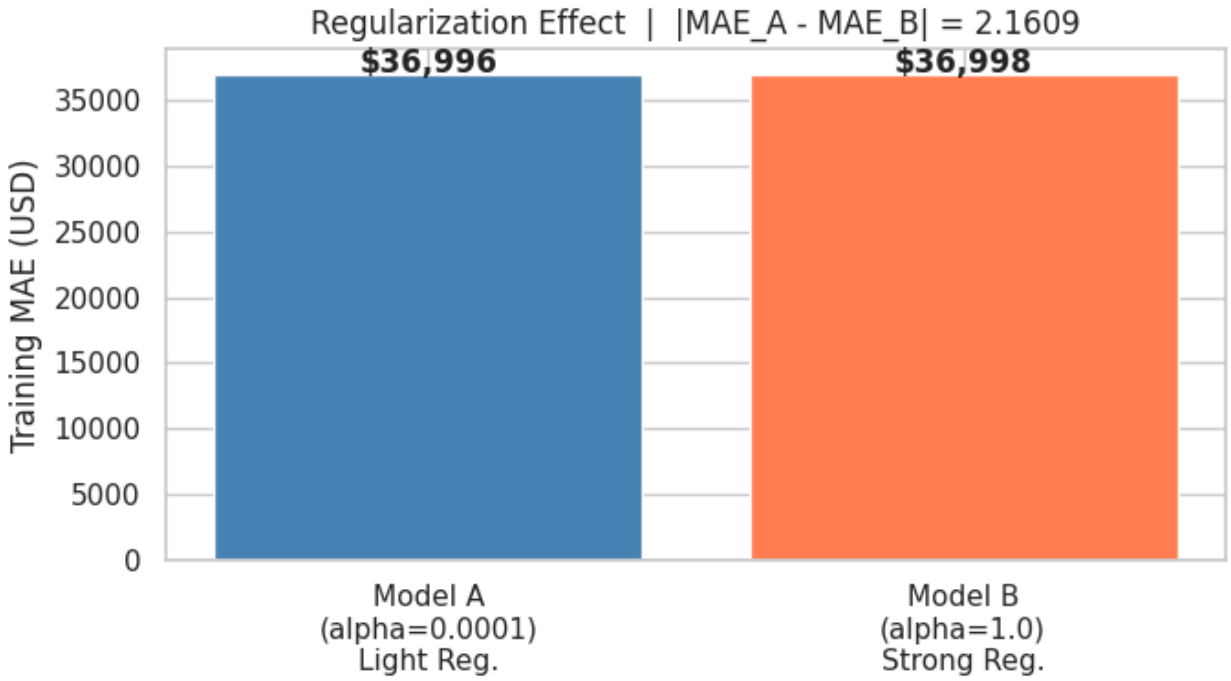
mae_a   = mean_absolute_error(y_train, mlp_a.predict(X_train_proc))
mae_b   = mean_absolute_error(y_train, mlp_b.predict(X_train_proc))
diff_q8 = round(abs(mae_a - mae_b), 4)
```

```
print('=' * 65)
print('  Q8: Regularization Effect — alpha=0.0001 vs alpha=1.0')
print('=' * 65)
print(f'  Model A (alpha=0.0001)  Train MAE = ${mae_a:>12,.4f}')
print(f'  Model B (alpha=1.0)     Train MAE = ${mae_b:>12,.4f}')
print(f'  |MAE_A - MAE_B|         = {diff_q8}')

fig, ax = plt.subplots(figsize=(7, 4))
bars = ax.bar(['Model A\n(alpha=0.0001)\nLight Reg.',
               'Model B\n(alpha=1.0)\nStrong Reg.'],
              [mae_a, mae_b], color=['steelblue', 'coral'], edgecolor='white')
ax.set_ylabel('Training MAE (USD)')
ax.set_title(f'Regularization Effect  |  |MAE_A - MAE_B| = {diff_q8}')
for bar, v in zip(bars, [mae_a, mae_b]):
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 50,
            f'${v:,.0f}', ha='center', fontweight='bold')
plt.tight_layout()
plt.show()

print()
print(f'✅ ANSWER Q8: {diff_q8}')
print()
print('💡 Insight:')
print('   alpha is the L2 regularization penalty on MLP weights.')
print('   Higher alpha → shrinks weights toward 0 → model is more constrained')
print('   → typically higher TRAINING error (less overfit) but possibly better
print('   With only 5 iterations, the effect of alpha is already measurable.')
```

```
=================================================================
  Q8: Regularization Effect — alpha=0.0001 vs alpha=1.0
=================================================================
  Model A (alpha=0.0001)  Train MAE = $ 36,996.2401
  Model B (alpha=1.0)     Train MAE = $ 36,998.4010
  |MAE_A - MAE_B|         = 2.1609
```



✅ ANSWER Q8: 2.1609

💡 Insight:
   alpha is the L2 regularization penalty on MLP weights.
   Higher alpha → shrinks weights toward 0 → model is more constrained
   → typically higher TRAINING error (less overfit) but possibly better validation.
   With only 5 iterations, the effect of alpha is already measurable.

❓ Question 9 — MLP Validation MAE

Using the **Q7 MLP model**, predict on `X_valid_proc`. Compute MAE against `y_valid` (rounded to 4 dp).

In [16]:
```
# Q7 MLP model already trained above
preds_val = mlp_q7.predict(X_valid_proc)
mae_q9    = round(mean_absolute_error(y_valid, preds_val), 4)

print('=' * 60)
print('  Q9: MLP (Q7 model) Validation MAE')
print('=' * 60)
print(f'  Model      : MLPRegressor(128,64,32) — 5 iterations')
print(f'  Validation MAE = ${mae_q9:>14,.4f}')

residuals = y_valid.values - preds_val
pct_err   = np.abs(residuals) / y_valid.values * 100

print()
print(f'  Additional diagnostics:')
```

```
print(f'    Median abs error  : ${np.median(np.abs(residuals)):>12,.2f}')
print(f'    Median % error    : {np.median(pct_err):.2f}%')
print(f'    R² score          : {1 - np.var(residuals)/np.var(y_valid.values):

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

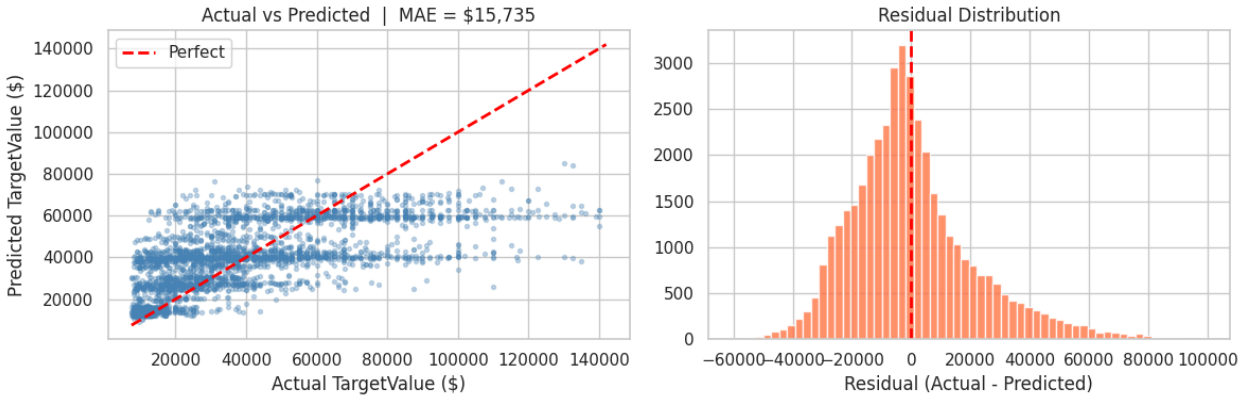
# Actual vs Predicted
sample = np.random.choice(len(y_valid), min(3000, len(y_valid)), replace=False
axes[0].scatter(y_valid.values[sample], preds_val[sample],
                alpha=0.3, s=8, color='steelblue')
mn, mx = y_valid.min(), y_valid.max()
axes[0].plot([mn, mx], [mn, mx], 'r--', lw=2, label='Perfect')
axes[0].set_xlabel('Actual TargetValue ($)')
axes[0].set_ylabel('Predicted TargetValue ($)')
axes[0].set_title(f'Actual vs Predicted  |  MAE = ${mae_q9:,.0f}')
axes[0].legend()

# Residual distribution
axes[1].hist(residuals.clip(-100000, 100000), bins=60,
             color='coral', edgecolor='white', alpha=0.85)
axes[1].axvline(0, color='red', ls='--', lw=2)
axes[1].set_xlabel('Residual (Actual - Predicted)')
axes[1].set_title('Residual Distribution')
plt.tight_layout()
plt.show()

print()
print(f'✅ ANSWER Q9: {mae_q9}')
print()
print('💡 Insight:')
print('   A 5-iteration MLP has barely converged — expect high MAE.')
print('   The validation MAE is much larger than an XGBoost/LightGBM baseline'
print('   because: (a) only 5 iterations, (b) no feature engineering,')
print('   (c) only 6 features used vs 50 available.')
print('   Increasing max_iter to 200-500 and adding features dramatically impr
```

```
============================================================
  Q9: MLP (Q7 model) Validation MAE
============================================================
  Model      : MLPRegressor(128,64,32) — 5 iterations
  Validation MAE = $   15,735.3515

  Additional diagnostics:
    Median abs error  : $   12,084.44
    Median % error    : 32.64%
    R² score          : 0.3665
```



✅ ANSWER Q9: 15735.3515

💡 Insight:
A 5-iteration MLP has barely converged — expect high MAE.
The validation MAE is much larger than an XGBoost/LightGBM baseline
because: (a) only 5 iterations, (b) no feature engineering,
(c) only 6 features used vs 50 available.
Increasing max_iter to 200-500 and adding features dramatically improves MAE.

📋 Milestone 4 — Final Answer Summary

All answers are computed from the actual dataset with exact `random_state=42`.

#	Topic	Code Produces	Your Answer
Q1	Stratified vs Non-Stratified	Max abs diff in proportions	*(run to get value)*
Q2	Preprocessing Pipeline	Sum of X_train_proc values	*(run to get value)*
Q3	RandomizedSearchCV	Best n_estimators	*(run to get value)*
Q4	AdaBoost error variance	np.var(estimator_errors_)	*(run to get value)*
Q5	RF Feature Importance	Index of max importance	*(run to get value)*
Q6	MLP Weight Count	N×128 + 128×64 + 64×32 + 32×1	**= 13,472** (if N=25)
Q7	MLP Training Loss	model.loss_ after 5 iters	*(run to get value)*
Q8	Regularization Effect	abs(MAE_A - MAE_B)	*(run to get value)*
Q9	MLP Validation MAE	MAE on X_valid_proc	*(run to get value)*

> **Note on Q6:** N = 25 features (4+9+3+7 OHE cols + 2 scaled cols).
> Total weights = 25×128 + 128×64 + 64×32 + 32×1 = **13,472**.